Componentes React

Programação Web 2

Objetivo de Aprendizagem

- Conhecer, criar e utilizar componentes React

Agenda

- O que são componentes React
- Funções vs Classes
- Props e passagem de dados
- Desestruturação de Props
- Props.children
- Exemplos práticos

Componentes React?

> São fragmentos independentes e reutilizáveis de código que tem o mesmo propósito que funções JS. Retornam elementos HTML através de uma função ou método *render*

```
1  function Car() {
2    return (
3      <h2>Eu sou um carro!</h2>
4    );
5  }
```

Tipos de Componentes

1. Function Components
2. Class Components

Tipos de Componentes

1. **Function Components** (Recomendado)

- Componentes criados como funções JavaScript
- Simples e diretos
- Suportam *Hooks*

Tipos de Componentes

2. **Class Components**

- Componentes criados como classes que estendem `React.Component`
- Mais complexos
- Usados em código legado

Regras Importantes

- Nome deve começar com LETRA MAIÚSCULA
  - `<Car />` ✓
  - `<car />` ✗
- Retornam HTML usando JSX
- Podem ser reutilizados

```
1  function Car() {
2    return <h2>Eu sou um carro!</h2>;
3  }
4
5  createRoot(document.getElementById('root')).render(<Car />);
```

Class Components

Criando um componente de classe

```
1  class Car extends React.Component {
2    render() {
3      return <h2>Oi, eu sou um Carro!</h2>;
4    }
5  }
```

Um componente de classe:

- Estende `React.Component`
- Possui um método `render()` obrigatório
- O método `render()` retorna HTML

Class Components

Constructor

> O constructor é chamado antes de qualquer outra coisa

```
1  class Car extends React.Component {
2    constructor(props) {
3      super(props);
4      this.state = { color: "red" };
5    }
6
7    render() {
8      return <h2>Eu sou um carro {this.state.color}!</h2>;
9    }
10 }
```

Class Components

Constructor

- Use `super(props)` para herdar as funções do `React.Component`
- Inicialize o `state` no constructor

Componentes dentro de Componentes

> Componentes podem ser usados dentro de outros componentes:

```
1   function Car() {
2     return <h2>Eu sou um Carro!</h2>;
3   }
4   function Garage() {
5     return (
6       <>
7         <h1>Quem mora em minha garagem?</h1>
8         <Car />
9       </>
10    );
11  }
12  createRoot(document.getElementById('root')).render(<Garage />);
```

Componentes em Arquivos Separados

> Organizar os componentes em arquivos separados (`.jsx`)

`vehicle.jsx`

```
1  import React from 'react';
2
3  function Car() {
4    return <h2>Oi, eu sou um Carro!</h2>;
5  }
6
7  export default Car;
```

`App.jsx`

```
1  import Car from './Vehicle.jsx';
2
3  createRoot(document.getElementById('root')).render(<Car />);
```

Props

> Props são como argumentos de função em JavaScript e atributos em HTML

```
1  function Car(props) {
2    return <h2>Eu sou um {props.color} Carro!</h2>;
3  }
4
5  createRoot(document.getElementById('root')).render(
6    <Car color="red" />
7  );
```

- Props são **read-only**
- Passadas como atributos no JSX
- Acessadas via `props.nomeDaPropriedade`

Props

Múltiplas Propriedades

```
1  function Car(props) {
2    return (
3      <h2>Eu sou um {props.color} {props.brand} {props.model}!</h2>
4    );
5  }
6
7  createRoot(document.getElementById('root')).render(
8    <Car brand="Ford" model="Mustang" color="red" />
9  );
```

> Cada atributo se torna uma propriedade no objeto `props`

Props

Diferentes Tipos de Dados

```
1  // Strings - aspas normais
2  <Car brand="Ford" />
3
4  // Números - chaves {}
5  <Car year={1969} />
6
7  // Variáveis - chaves {}
8  let x = "Ford";
9  <Car brand={x} />
10
11 // Objetos e Arrays - chaves {}
12 let carInfo = { name: "Ford", model: "Mustang" };
13 <Car carinfo={carInfo} />
```

Props

Acessando Objetos

```
1  function Car(props) {
2    return (
3      <>
4        <h2>Meu {props.carinfo.name} {props.carinfo.model}!</h2>
5        <p>Cor: {props.carinfo.color}</p>
6      </>
7    );
8  }
```

Props

Acessando Arrays

```
1  function Car(props) {
2    return (
3      <h2>Meu carro é um {props.carinfo[0]} {props.carinfo[1]}!</h2>
4    );
5  }
```

Props

Parâmetros entre Componentes

```
1   function Car(props) {
2     return <h2>Eu sou um {props.brand}!</h2>;
3   }
4
5   function Garage() {
6     return (
7       <>
8         <h1>Quem mora em minha garagem?</h1>
9         <Car brand="Ford" />
10      </>
11    );
12  }
13
14  createRoot(document.getElementById('root')).render(<Garage />);
```

Desestruturação de Props

> Visa simplificar o código desestruturando o que for necessário

```
1   // Sem desestruturação
2   function Car(props) {
3     return <h2>Meu carro é {props.color}!</h2>;
4   }
5
6   // Com desestruturação - no parâmetro
7   function Car({color}) {
8     return <h2>Meu carro é {color}!</h2>;
9   }
10
11  // Com desestruturação - no corpo da função
12  function Car(props) {
13    const {brand, model} = props;
14    return <h2>Eu amo meu {brand} {model}!</h2>;
15  }
```

Desestruturação

Rest Operator

> Utilizar `... rest` quando a quantidade `props` é desconhecida

```
function Car({color, brand, ...rest}) {
  return (
    <h2>Meu {brand} {rest.model} é {color}!</h2>
  );
}
<Car
  brand="Ford"
  model="Mustang"
  color="red"
  year={1969}
/>
```

- Nesse exemplo `rest` armazena: `{ model: "Mustang", year: 1969 }`

Desestruturação

Valores Padrão

> É possível configurar valores padrão para `props``

```
1  function Car({color = "blue", brand}) {
2    return (
3      <h2>Meu {color} {brand}!</h2>
4    );
5  }
6
7  <Car brand="Ford" />
```

- Se a `prop` não for fornecida, o valor padrão é utilizado

`Props.children`

> Utilizar `props.children` para enviar com conteúdo entre tags de abertura e fechamento (ver exemplo)

```
function Son(props) {
  return (
    <div style={{background: 'lightgreen'}}>
      <h2>Son</h2>
      <div>{props.children}</div>
    </div>
  );
}
function Daughter(props) {
  const {brand, model} = props;
  return (
    <div style={{background: 'lightblue'}}>
      <h2>Daughter</h2>
      <div>{props.children}</div>
    </div>
  );
}
function Parent() {
  return (
    <div>
      <h1>My two Children</h1>
      <Son>
        <p>
          This was written in the Parent component,
          but displayed as a part of the Son component
        </p>
      </Son>
```

Props.children

Múltiplos Componentes

```
1   function Daughter(props) {
2     return (
3       <div style={{background: 'lightblue'}}>
4         <h2>Filha</h2>
5         <div>{props.children}</div>
6       </div>
7     );
8   }
9
10  function Parent() {
11    return (
12      <>
13        <Son>
14          <p>Conteúdo do Filho</p>
15        </Son>
16        <Daughter>
17          <p>Conteúdo da Filha</p>
18        </Daughter>
19      </>
20    );
21  }
```

State vs Props

Aspecto	Props	State
Origem	Recebidas como parâmetros	Definidas no próprio componente
Leitura	Read-only (imutável)	Pode ser alterado
Use para	Passar dados entre componentes	Dados que mudam
Atualização	Não pode ser alterada	`setState()` em classes

State em Class Components

```
1   class Car extends React.Component {
2     constructor(props) {
3       super(props);
4       this.state = {
5         brand: "Ford",
6         color: "red",
7         year: 1964
8       };
9     }
10
11    render() {
12      return (
13        <>
14          <h1>Meu {this.state.brand}</h1>
15          <p>Cor: {this.state.color}, Ano: {this.state.year}</p>
16        </>
17      );
18    }
19  }
```

Alterando State em Class Components

```
1   class Car extends React.Component {
2     constructor(props) {
3       super(props);
4       this.state = { color: "red" };
5     }
6
7     changeColor = () => {
8       this.setState({ color: "blue" });
9     }
10
11    render() {
12      return (
13        <>
14          <h2>Meu carro é {this.state.color}</h2>
15          <button onClick={this.changeColor}>Mudar Cor</button>
16        </>
17      );
18    }
19  }
```

- Use `setState()` para atualizar state, não faça atribuição direta!

Exemplo 1

Card de Produto

```
function ProductCard({name, price, image, children}) {
  return (
    <div style={{border: '1px solid #ccc', padding: '10px'}}>
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>R$ {price}</p>
      <div>{children}</div>
    </div>
  );
}

// Uso
<ProductCard
  name="Notebook"
  price="2999.99"
  image="notebook.jpg"
>
  <button>Comprar</button>
</ProductCard>
```

Exemplo 2

Lista de Itens

```
1   function Item({title, completed}) {
2     return (
3       <li style={{
4         textDecoration: completed ? 'line-through' : 'none'
5       }}>
6         {title}
7       </li>
8     );
9   }
10
11  function ItemList({items}) {
12    return (
13      <ul>
14        {items.map((item, index) => (
15          <Item key={index} title={item.title} completed={item.completed} />
16        ))}
17      </ul>
18    );
19  }
```

Boas práticas

- Use nomes auto-descritivos para props
- Desestruture props quando possível
- Mantenha componentes simples
- Reutilize componentes o máximo possível
- Utilizar `Props.children` para composição
- Evitar props muito aninhadas (prop drilling)

Exemplo 3

Componente de Botão

```
function Button({text, onClick, color = "blue"}) {
  return (
    <button
      onClick={onClick}
      style={{
        backgroundColor: color,
        color: 'white',
        padding: '10px 20px',
        border: 'none',
        borderRadius: '4px',
        cursor: 'pointer'
      }}
    >
      {text}
    </button>
  );
}

// Uso
<Button text="Clique aqui" color="green" onClick={() => alert('Clicado!')} />
```

Exemplo 4

Componente de Card

```
function Card({title, description, footer}) {
  return (
    <div style={{
      border: '1px solid #ddd',
      borderRadius: '8px',
      padding: '16px',
      marginBottom: '16px',
      boxShadow: '0 2px 8px rgba(0,0,0,0.1)'
    }}>
      <h2>{title}</h2>
      <p>{description}</p>
      {footer && <footer>{footer}</footer>}
    </div>
  );
}

// Uso
<Card
  title="Bem-vindo"
  description="Este é um componente de card"
  footer="Rodapé do card"
/>
```

Exemplo 5

Componente com Composição

```
function Container({title, children}) {
  return (
    <div style={{
      maxWidth: '600px',
      margin: '0 auto',
      padding: '20px',
      backgroundColor: '#f5f5f5'
    }}>
      {title && <h1>{title}</h1>}
      <div>{children}</div>
    </div>
  );
}

// Uso
<Container title="Meu Conteúdo">
  <p>Conteúdo dentro do container</p>
  <Button text="Enviar" />
</Container>
```

Perguntas?

Exercícios

1

Criar um componente `Greeting` que:

- Aceita uma prop `name`
- Retorna `<h1>Olá, {name}!</h1>`
- Use-o para saudar 3 pessoas diferentes

2

Criar um componente `PersonCard` que:

- Aceita props: `name`, `age`, `city`
- Exibe as informações formatadas
- Use desestruturação de props

3 (`prop.children`)

Criar um componente `Alert` que:

- Aceita uma prop `type` ("success", "error", "warning")
- Usa `props.children` para o conteúdo
- Muda a cor de fundo baseado no tipo

4 (Componente de Lista)

Crie um componente `TodoList` que:

- Aceita uma prop `items` (array de objetos)
- Cada item tem `id` e `text`
- Renderiza como lista `<li>`

```
1   // Sua solução aqui
2   function TodoList({items}) {
3     // Use map para renderizar
4     // ...
5   }
6
7   // Teste com:
8   const todos = [
9     {id: 1, text: "Aprender React"},
10    {id: 2, text: "Criar componentes"},
11    {id: 3, text: "Dominar Props"}
12  ];
13  // <TodoList items={todos} />
```

5 (Componente de Formulário)

Criar um componente `FormInput` que:

- Aceita props: `label`, `type`, `placeholder`, `value`, `onChange`
- Renderiza um label e um input
- Use desestruturação com `...rest` para aceitar outras props do input

Referências

- React Documentation
- W3Schools React Components
- React DevTools

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

pw2-repo

Powered by Slidev

Cover image by haikei